

Orbit Wars: Các phương pháp cho trò chơi chinh phục hành tinh thời gian thực

Nhóm số: 11

Nguyễn Trường An – MSSV: 23020323

Hoàng Ngọc Điệp – MSSV: 23020357

Viện trí tuệ nhân tạo

Trường Đại học Công nghệ, ĐHQGHN

Email: {23020323, 23020357}@vnu.edu.vn

Abstract—Bài báo cáo trình bày hai cách tiếp cận độc lập của Nhóm 11 trong cuộc thi Kaggle Orbit Wars – trò chơi chinh phục hành tinh thời gian thực được mô hình hóa như bài toán học tăng cường đa tác nhân. Pipeline thứ nhất (Hoàng Ngọc Điệp) áp dụng học tăng cường ngoại tuyến: từ Bắt chước Hành vi (BC) đến IQL+CQL với suy luận ensemble bị quan. Pipeline thứ hai (Nguyễn Trường An) xây dựng hệ thống lập kế hoạch dựa trên mô hình thế giới với cơ học quỹ đạo và chấm điểm nhiệm vụ chuẩn hóa Z-score, sau đó tiền hóa thành hệ thống lập kế hoạch tensor hóa (tensorized planning) với chấm điểm đối kháng (competitive scoring) và mạng nơ-ron, phát triển liên tục qua 25 phiên bản.

I. GIỚI THIỆU

Orbit Wars là một trò chơi chiến thuật thời gian thực lấy bối cảnh không gian vũ trụ, được tổ chức dưới dạng một cuộc thi lập trình trên nền tảng Kaggle. Khác với các trò chơi cờ như Cờ vua, Cờ vây vốn có không gian trạng thái rời rạc và tĩnh, Orbit Wars sở hữu một môi trường động lực học phức tạp, trong đó từ 2 đến 4 người chơi (agents) cạnh tranh với nhau để giành quyền kiểm soát một mạng lưới các hành tinh.

Mỗi hành tinh trong hệ thống không chỉ đứng yên mà còn có thể quay quanh một mặt trời trung tâm. Trò chơi diễn ra trong tối đa 500 lượt. Nếu sau 500 lượt có nhiều hơn một người chơi sống sót, người chiến thắng sẽ được quyết định dựa trên tổng số lượng tàu vũ trụ đang sở hữu. Tại mỗi lượt, các hành tinh tự động sản sinh ra một số lượng tàu vũ trụ nhất định dựa trên production rate. Người chơi có thể ra lệnh cho các hành tinh mà mình kiểm soát phóng một phần hoặc toàn bộ số tàu đồn trú (garrison) đến các hành tinh khác. Mục đích của các chuyến bay này có thể là tiếp viện cho đồng minh, phòng thủ, hoặc tấn công chiếm đóng hành tinh của đối phương hay hành tinh trung lập.

Nhóm áp dụng chiến lược *song song độc lập*: Hoàng Ngọc Điệp phát triển pipeline offline RL, Nguyễn Trường An xây dựng pipeline lập kế hoạch dựa trên mô hình thế giới qua 25 phiên bản. Bài nộp tốt nhất đạt **1146.6 điểm công khai** tại phiên bản **v24**.

A. Danh sách thành viên

TABLE I: Thành viên nhóm

Họ và tên	MSSV	Vai trò
Nguyễn Trường An	23020323	Đại diện, Agent V1–V25
Hoàng Ngọc Điệp	23020357	Offline RL pipeline, Báo cáo

B. Phân công công việc

Nhóm áp dụng chiến lược *song song độc lập*: mỗi thành viên tự thiết kế và phát triển agent riêng, cuối cùng chọn agent tốt nhất để đại diện nhóm submit.

Nguyễn Trường An:

- Thiết kế và xây dựng kiến trúc WorldModel + Mission từ đầu (v1), liên tục cải tiến qua 25 phiên bản (v1–v25).
- Thêm các tính năng then chốt: an toàn quỹ đạo (v2), di tản sao chổi (v5), chế độ 1v1 (v11), nhiệm vụ STEAL/STAGED_RELAY (v14), bộ lọc anti-vulture (v16–v17), tối ưu tham số đa giai đoạn (v18–v23).
- Chuyển đổi sang kiến trúc tensor hóa (v24): sử dụng thư viện *orbit_lite* tự xây dựng, áp dụng hàm chấm điểm đối kháng dựa trên Flow-diff projection.
- Thử nghiệm Mạng nơ-ron chấm điểm nhiệm vụ (v25): sử dụng MLP để dự đoán xác suất thành công của các ứng viên nhiệm vụ.
- Submit và theo dõi kết quả trên Kaggle; đạt điểm công khai tốt nhất **1146.6** (v24), xếp hạng **625**.
- Đánh giá kết quả thực nghiệm và phân tích hiệu quả từng phiên bản.

Hoàng Ngọc Điệp:

- Nghiên cứu và triển khai pipeline offline RL (Behavioral Cloning, IQL+CQL) qua các phiên bản v5–v8.
- Thiết kế MDP formulation, shaped reward, và thu thập dữ liệu self-play.
- Huấn luyện mô hình MLP và ensemble Q-network trên Kaggle Notebook.
- Phân tích lỗi kỹ thuật (weight corruption bug) và đề xuất fix.
- Viết báo cáo.

II. LINK GITHUB

Source code của nhóm được lưu trữ tại: [//github.com/IAI-RL-Cherry_11/orbit-wars //](https://github.com/IAI-RL-Cherry_11/orbit-wars)

Repository bao gồm: source code các phiên bản agent của Nguyễn Trường An, source code pipeline offline RL của Hoàng Ngọc Diệp, và báo cáo nhóm.

III. KẾT QUẢ VÀ RANKING

A. Điểm bảng xếp hạng

- **Điểm công khai tốt nhất: 1146.6** (v24 – Nguyễn Trường An)
- **Thứ hạng tốt nhất: 625**

IV. CÁCH TIẾP CẬN VÀ KẾT QUẢ 1

A. Phát Biểu Bài Toán

Orbit Wars được mô hình hóa như một Markov Decision Process (MDP) với:

- **Trạng thái s_t :** Tập hợp trạng thái các hành tinh $\{(id, owner, x, y, r, ships, prod)\}$, đội hình tàu đang bay $\{fleet\}$, lượt hiện tại t , và vận tốc góc quỹ đạo ω .
- **Hành động a_t :** Tập hợp lệnh gửi tàu $\{(source_id, angle, ships)\}$, tối đa 10 lệnh mỗi lượt. Không gian hành động có kích thước biến đổi theo trạng thái.
- **Phần thưởng r_t :**

$$r_t = \lambda \cdot \Delta \left(\frac{P_{my}}{P_{total}} \right) + \mathbb{I}[terminal] \cdot R_{win/loss} \quad (1)$$

trong đó P_{my} , P_{total} là sản xuất của agent và tổng toàn bản đồ, $\lambda = 8$, $R_{win} = +50$, $R_{loss} = -50$.

- **Hệ số chiết khấu:** $\gamma = 0.99$.
- **Chân trời thời gian:** $T = 500$ lượt.

B. Thách thức kỹ thuật

Bài toán đặt ra nhiều thách thức đặc trưng:

(1) **Phân bổ công nhận (Credit assignment):** Với $T = 500$ lượt, quyết định ở lượt đầu ảnh hưởng đến kết quả cuối nhưng tín hiệu phần thưởng thưa.

(2) **Không gian hành động biến đổi:** Số lượng mục tiêu khả dụng thay đổi theo mỗi lượt, không thể dùng kiến trúc mạng nơ-ron cố định thông thường.

(3) **Tính không dừng đa tác nhân:** Đối thủ cũng cập nhật chiến lược, khiến môi trường không ổn định từ góc nhìn của một tác nhân đơn lẻ.

(4) **Ràng buộc ngoại tuyến:** Không được tương tác thêm với môi trường sau khi nộp bài; tất cả việc học phải từ dữ liệu đã thu thập trước.

C. Phiên bản v5: Heuristic + Bắt Chước Hành Vi

Kiến trúc: Mạng nơ-ron đa lớp (MLP) 4 lớp ($13 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$) được huấn luyện theo phương pháp Bắt chước Hành vi (BC) với 200 episode tự chơi.

Hàm chấm điểm heuristic (dùng làm dự phòng và nhân huấn luyện trong BC):

$$s(t) = P_{val} + B_{owner} + B_{close} - c_{ships} - c_{req} - c_{eta} \quad (2)$$

trong đó $P_{val} = prod \times \min(T_{left}, 120) \times 0.25$, B_{owner} là phần thưởng cộng thêm theo loại hành tinh (trung lập hay địch).

Vấn đề phát hiện: Trọng số xuất ra từ MLPRegressor của sklearn có nhiều giá trị là số thực không chuẩn hóa ($\sim 10^{-315}$), thực tế bằng 0. Nguyên nhân: ma trận đầu ra W_3 có hình dạng (1,16) thay vì (16,1), gây lỗi `TypeError: only 0-dimensional arrays can be converted to Python scalars` khi nộp bài.

Kết quả: Điểm 536.5

D. Phiên bản v6: Sửa Lỗi + Cải Thiện Heuristic

Sửa lỗi mạng: Thay `float(W3 @ x + b3)` bằng `float(np.asarray(W3 @ x + b3).flat[0])` để xử lý đầu ra có hình dạng (1,).

Cải tiến chiến lược:

- **Dự trữ thích nghi:** f_{send} thay đổi theo giai đoạn trò chơi và tỷ lệ sản xuất (0.85 đầu, 0.90 giữa-thắng, 0.95 cuối).
- **Phân công:** Phát hiện hành tinh địch vừa gửi quân ra (quân đồn trú thấp hơn $8 \times prod$) và ưu tiên tấn công.
- **Điểm thưởng hành tinh địch:** $B_{enemy} = 4.0 + prod \times 2.0$ để phản ánh giá trị “chiếm + tước đoạt” trong đối đầu 1v1.
- **Đánh giá giá trị trạng thái:** Hệ số nhân giai đoạn cuối $\times 1.5$ cho đánh giá nhìn trước.

Kết quả: Điểm 542.6 (+6.1 so với v5).

E. Phiên bản v7: Heuristic ROI Thuần + Chế Độ Tiêu Diệt

Nhận thấy trọng số mạng vẫn bị nhiễu (tự chơi với heuristic yếu), v7 loại bỏ hoàn toàn mạng nơ-ron và thay bằng công thức Lợi nhuận trên Đầu tư (ROI) có nền tảng lý thuyết hơn:

$$ROI(s, a) = \frac{\Delta P \times T_{eff}}{C_{ships}} - d \cdot \text{dist} \quad (3)$$

trong đó $\Delta P = prod$ (trung lập) hoặc $2 \times prod$ (địch – phản ánh cả phần được và tước đoạt của đối thủ), $T_{eff} = \min(T_{left} - \eta, 400)$, C_{ships} là số tàu cần gửi.

Chế độ Tiêu diệt: Khi $ratio_{prod} \geq 1.3$ hoặc giai đoạn cuối, tấn công *tất cả* hành tinh địch đồng thời (tối đa 8 cuộc/lượt) để ngăn địch phân bổ lại quân phòng thủ.

Chế độ Xông lên: 70 lượt đầu, chỉ tấn công hành tinh trung lập – tối đa hóa số hành tinh chiếm được sớm.

Tối ưu biên độ: Biên độ overkill giảm từ $\max(8, prod \times 3)$ xuống $\max(2, prod \times 1.2)$, tiết kiệm 5–10 tàu mỗi cuộc tấn công.

F. Phiên bản v8: IQL + CQL + Tập hợp Bi quan

Đây là phiên bản áp dụng học tăng cường ngoại tuyến hiện đại nhất.

1) **Thu thập dữ liệu:** 400 episode tự chơi với agent v7 (mạnh hơn v5), ghi nhật ký đầy đủ:

- **Đặc trưng trạng thái s_t** (10 chiều): tỷ lệ sản xuất, tỷ lệ tàu, tỷ lệ hành tinh, thời gian, mật độ trung lập, tỷ lệ thống trị, tỷ lệ quân đang bay, vùng địch, cờ giai đoạn cuối, tỷ lệ sản xuất trung lập.

TABLE II: Kết quả các phiên bản agent trên Bảng Xếp Hạng Công Khai

Phiên bản	Mô tả	Công khai	Tốt nhất
<i>Hoàng Ngọc Diệp – Học tăng cường ngoại tuyến</i>			
v5	Heuristic + BC (lỗi trọng số)	536.5	536.5
v6	BC sửa lỗi + Heuristic nâng cao	542.6	542.6
v7	ROI thuần + Chế độ Tiêu diệt	589.0	589.0
v8	IQL+CQL+Ensemble + ROI	543.0	543.0
<i>Nguyễn Trường An – Mô hình thể giới & Lập kế hoạch Tensor (v1–v25)</i>			
v1	WorldModel + Mission + Knapsack	793.8	793.8
v2	+ An toàn quỹ đạo, RELAY, Comet	–	–
v5	+ Di tản sao chổi (comet evac)	–	–
v11	+ Chế độ 1v1	–	–
v14	+ STEAL, STAGED_RELAY, RELAY_LOCK	–	–
v16	+ Bộ lọc anti-vulture	–	–
v17	+ Anti-vulture nâng cao	–	–
v18–v22	Tối ưu tham số đa giai đoạn	886.6	886.6
v23	Tham số tối ưu cuối (Heuristic)	889.0	889.0
v24	Bài nộp tốt nhất (nhóm) – Tensorized	1146.6	1146.6
v25	+ Neural Network Scoring	1080.8	1080.8

– = Không có dữ liệu submit riêng lẻ.

- **Đặc trưng hành động** (s, a) (27 chiều): 17 chiều hành động (mở rộng từ 13 của v5 bằng 4 đặc trưng mới: thống trị, lợi thế tàu, áp lực bản đồ, tuổi quân đồn trú) ghép với 10 chiều trạng thái.
- Phần thưởng mỗi lượt: $r_t = \lambda \cdot \Delta(P_{my}/P_{total})$.

2) *Học Q Ẩn (IQL)*: IQL [1] tránh tính $Q(s, a')$ cho toàn bộ hành động (vấn đề khi không gian hành động biến đổi) bằng cách học riêng $V(s)$ qua hồi quy expectile:

$$\mathcal{L}_V = \mathbb{E} \left[L_\tau \left(\min_i Q_i(s, a) - V(s) \right) \right] \quad (4)$$

$$L_\tau(u) = |\tau - \mathbf{1}(u < 0)| \cdot u^2, \quad \tau = 0.7 \quad (5)$$

V được huấn luyện như bộ ước lượng lạc quan ($\tau > 0.5$) của Q . Mạng Q được cập nhật bằng học TD với V làm đích bootstrap:

$$\mathcal{L}_Q = \mathbb{E} \left[(r_t + \gamma V(s_{t+1}) - Q(s_t, a_t))^2 \right] \quad (6)$$

3) *Học Q Bảo Thủ (CQL)*: Hình phạt CQL [2] ngăn ước lượng quá cao Q cho các hành động ngoài phân phối dữ liệu huấn luyện:

$$\mathcal{L}_{CQL} = \alpha \left(\log \sum_a \exp Q(s, a) - Q(s, a_{behavior}) \right) \quad (7)$$

Hàm mất mát huấn luyện cuối: $\mathcal{L} = \mathcal{L}_Q + \alpha \cdot \mathcal{L}_{CQL}$, với $\alpha = 0.5$.

4) *Suy Luận Tập Hợp Bi Quan*: Huấn luyện $N = 3$ mạng Q độc lập. Khi suy luận, sử dụng:

$$Q_{bi-quan}(s, a) = \min_{i=1}^3 Q_i(s, a) \quad (8)$$

Phương pháp này [3] tránh khai thác các vùng không chắc chắn của hàm Q , đặc biệt quan trọng trong bối cảnh học ngoại tuyến.

5) *Trích Xuất Chính Sách*: Trích xuất chính sách qua Hồi quy Có Trọng số Lợi thế [4]:

$$w(s, a) = \exp \left(\frac{Q_{bi-quan}(s, a) - V(s)}{\beta} \right), \quad \beta = 1.0 \quad (9)$$

Điểm cuối để xếp hạng các mục tiêu ứng viên:

$$\text{điểm}(s, a) = 0.5 \cdot Q_{bi-quan}(s, a) + 0.5 \cdot \text{ROI}(s, a) \quad (10)$$

Kết hợp 50/50 đảm bảo nếu mạng chưa học tốt, heuristic v7 vẫn duy trì hiệu quả tối thiểu.

V. CÁCH TIẾP CẬN VÀ KẾT QUẢ 2

Thay vì dùng học tăng cường thuần, em xây dựng hệ thống *lập kế hoạch dựa trên mô hình*, đã phát triển qua 25 phiên bản với các milestone kiến trúc sau đây.

A. CƠ HỌC QUỸ ĐẠO VÀ ĐỊNH NGHĨA KHÔNG GIAN TOÁN HỌC

Để hệ thống máy tính có thể lập kế hoạch hiệu quả, bước đầu tiên và quan trọng nhất là phải mã hóa luật chơi và các định luật vật lý giả lập của Orbit Wars thành các phương trình toán học nghiêm ngặt. Chương này trình bày cách định nghĩa không gian trạng thái, mô hình hóa động học của các thiên thể, và thuật toán tính toán quỹ đạo đánh chặn.

1) *Cấu Trúc Không Gian Trạng Thái (State Space $\mathcal{S}$)*: Tại bất kỳ thời điểm (lượt) t nào, trạng thái toàn cục của hệ thống $s_t \in \mathcal{S}$ được định nghĩa bởi hai tập hợp chính: Tập hợp các hành tinh $\mathcal{P}$ và Tập hợp các hạm đội đang bay $\mathcal{F}$.

Mỗi hành tinh $p \in \mathcal{P}$ được đặc trưng bởi một vector trạng thái:

$$p = \langle x, y, r, \theta, \omega, o, g, \rho \rangle$$

Trong đó:

- (x, y) : Tọa độ Descartes 2D hiện tại.
- r : Bán kính quỹ đạo so với tâm hệ mặt trời.

- θ : Pha góc hiện tại (radians).
- ω : Vận tốc góc.
- $o \in \{-1, 0, 1, 2, 3\}$: Định danh chủ sở hữu (Owner ID), với -1 là trung lập (neutral).
- $g \in \mathbb{N}$: Số lượng tàu đang đồn trú (Garrison).
- $\rho \in \mathbb{R}^+$: Tốc độ sản xuất tàu tự động (Production rate).

Mỗi hạm đội $f \in \mathcal{F}$ đang bay lơ lửng trong không gian được đặc trưng bởi:

$$f = \langle x, y, o, s, src, tgt \rangle$$

Trong đó:

- (x, y) : Tọa độ hiện tại của hạm đội.
- o : Định danh chủ sở hữu.
- s : Số lượng tàu trong hạm đội.
- src, tgt : ID của hành tinh xuất phát và hành tinh đích đến.

2) *Cơ học quỹ đạo hành tinh (Orbital Kinematics)*: Ngoại trừ các sao chổi di chuyển theo vị trí được cung cấp sẵn và biến mất khi bay ra khỏi bản đồ, tất cả các hành tinh đều quay quanh một mặt trời trung tâm tọa lạc tại $C = (50.0, 50.0)$ với bán kính và chu kỳ quỹ đạo $R_{sun} = 10.0$.

Dựa vào các tham số trong trạng thái của hành tinh p , ta có thể dự đoán chính xác tọa độ của nó tại bất kỳ thời điểm tương lai $t_{future} = t + \Delta t$ bằng phương trình lượng giác:

$$x_p(t_{future}) = 50.0 + r \cdot \cos(\theta + \omega \cdot \Delta t) \quad (11)$$

$$y_p(t_{future}) = 50.0 + r \cdot \sin(\theta + \omega \cdot \Delta t) \quad (12)$$

Tính chất tuần hoàn của quỹ đạo này tạo ra một hiện tượng thú vị trong Orbit Wars: Một hành tinh đang ở phe ta có thể đang bay lại gần căn cứ địch, biến nó thành một bàn đạp lý tưởng, hoặc ngược lại, một hành tinh đang ở sâu trong lãnh thổ địch có thể đang quay dần về phía lãnh thổ của ta.

3) *Động học hạm đội (Fleet Dynamics)*: Khác với hành tinh di chuyển theo quỹ đạo tròn, một khi hạm đội được phóng đi, nó sẽ di chuyển theo một đường thẳng từ điểm xuất phát đến vị trí đánh chặn.

Điểm làm nên chiều sâu chiến thuật của trò chơi là: **Tốc độ của hạm đội tỷ lệ nghịch với quy mô của nó**. Một hạm đội nhỏ bay rất nhanh, tạo lợi thế bất ngờ. Một hạm đội lớn mang sức mạnh hủy diệt nhưng di chuyển chậm chạp, cho phép đối phương có thời gian nhận diện và gửi quân tiếp viện để phòng thủ. Đây là lý do khi ta quan sát trận đấu của các top tier, nhất là top 1, họ thường chia nhỏ quân thành số lượng hạm đội tấn công rất lớn để tận dụng tối đa tốc độ áp sát.

Hàm tốc độ v_f phụ thuộc vào số lượng tàu s được tính bằng phương trình phi tuyến:

$$v_f(s) = V_{max} - (V_{max} - 1.0) \cdot \left[\min \left(1.0, \frac{\log(\max(s, 1))}{\log(S_{cap})} \right) \right]^{1.5} \quad (13)$$

Trong đó, $V_{max} = 6.0$ là vận tốc giới hạn trên, và $S_{cap} = 1000$ là ngưỡng số lượng tàu mà tại đó vận tốc chạm mức tối thiểu ($v_f = 1.0$). Hệ số hàm mũ 1.5 tạo ra độ dốc lớn, khiến cho việc chia nhỏ hạm đội đôi khi mang lại lợi ích khổng lồ về mặt thời gian.

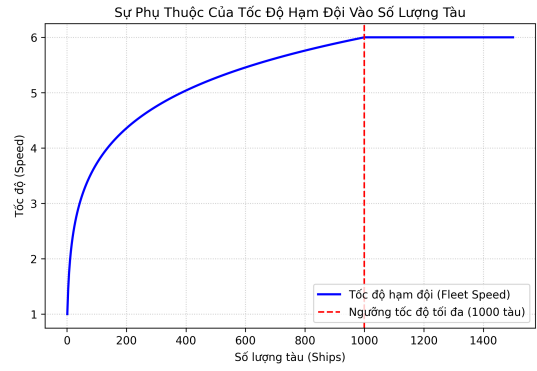


Fig. 1: Phân phối tốc độ hạm đội theo số lượng tàu. Đồ thị minh họa sự sụt giảm tốc độ mạnh mẽ khi quy mô hạm đội tăng lên từ 1 đến 300 tàu.

4) *Bài toán đánh chặn mục tiêu động*: Do đích đến là một hành tinh di chuyển liên tục theo phương trình (11) và (12), hạm đội không thể nhắm trực tiếp vào tọa độ hiện tại của hành tinh. Thay vào đó, hệ thống phải dự đoán vị trí tương lai của mục tiêu và tìm thời gian đánh chặn nhỏ nhất $\Delta t > 0$ sao cho hạm đội có thể gặp hành tinh tại đúng thời điểm đó.

Điều kiện đánh chặn được mô tả bởi phương trình:

$$\left\| \begin{bmatrix} 50 + r_{tgt} \cos(\theta_{tgt} + \omega_{tgt} \Delta t) \\ 50 + r_{tgt} \sin(\theta_{tgt} + \omega_{tgt} \Delta t) \end{bmatrix} - \mathbf{P}_{src} \right\| = v_f(s) \cdot \Delta t \quad (14)$$

trong đó về trái là khoảng cách hình học từ vị trí xuất phát đến vị trí tương lai của hành tinh tại thời điểm Δt , còn về phải là quãng đường tối đa mà hạm đội có thể di chuyển trong cùng khoảng thời gian.

Phương trình (14) là một phương trình siêu việt do chứa đồng thời các hàm lượng giác và biến thời gian Δt , nên không tồn tại nghiệm giải tích dạng đóng (closed-form solution). Để đáp ứng giới hạn thời gian xử lý của môi trường thi đấu, em sử dụng thuật toán quét số học (Numerical Scanning) nhằm tìm nghiệm gần đúng đầu tiên của phương trình với độ phức tạp thời gian $\mathcal{O}(H_{max})$:

Algorithm 1 Quét Đánh Chặn (Intercept Aiming Scan)

```

1: Input:  $\mathbf{P}_{src}$ , hành tinh đích  $Tgt$ , số tàu  $s$ 
2:  $v_f \leftarrow \text{fleet\_speed}(s)$ 
3: for  $t_{arr} = 1$  to 150 do
4:    $\mathbf{P}_{future} \leftarrow \text{predict\_planet}(Tgt, t_{arr})$ 
5:    $d \leftarrow \text{EuclideanDistance}(\mathbf{P}_{src}, \mathbf{P}_{future})$ 
6:    $t_{req} \leftarrow \lceil d/v_f \rceil$ 
7:   if  $t_{req} \leq t_{arr}$  then
8:     Return  $t_{arr}$ ,  $\text{VectorDirection}(\mathbf{P}_{src}, \mathbf{P}_{future})$ 
9:   end if
10: end for
11: Return Không thể đánh chặn

```

5) *Định tuyến tránh chướng ngại vật (Obstacle Bypass Routing)*: Ngay cả khi tìm được quỹ đạo đánh chặn theo

Algorithm 1, quỹ đạo đó (đoạn thẳng từ $\mathbf{P}_{src}$ đến $\mathbf{P}_{future}$) có thể đi xuyên qua mặt trời hoặc một hành tinh khác, khiến hạm đội bị tiêu diệt.

Để kiểm tra an toàn, em tính toán khoảng cách hình học ngắn nhất từ tâm mặt trời $\mathbf{C} = (50, 50)$ đến đoạn thẳng quỹ đạo bằng phép chiếu vector hay còn gọi là tính khoảng cách từ điểm đến 1 đường thẳng. Nếu khoảng cách này nhỏ hơn $R_{sun} + margin$, quỹ đạo bị chặn.

Khi đường bay bị chặn, hệ thống kích hoạt tính năng rẽ nhánh thông qua hàm `bypass_angle`. Nguyên lý của hàm này là xác định hình nón tạo bởi điểm xuất phát và chướng ngại vật, sau đó tính toán hai tiếp tuyến của đường tròn an toàn bao quanh chướng ngại vật:

$$\alpha_{half} = \arcsin \left(\frac{R_{obs} + margin}{\|\mathbf{P}_{src} - \mathbf{C}\|} \right) \quad (15)$$

Hệ thống sẽ thử hai góc $\theta_{direct} \pm \alpha_{half}$ để tìm đường bay lách qua mép mặt trời an toàn nhất. Đoạn mã lõi của thuật toán này đã được công bố tại phần Phụ lục v24.

B. WORLD MODEL VÀ LOOKAHEAD

Sau khi đã làm chủ được cơ học không gian và các hàm định tuyến, nhiệm vụ tiếp theo của `WorldModel` là dự báo tương lai của trận đấu. Trong một trò chơi RTS, câu hỏi quan trọng nhất mà bất kỳ chiến lược gia nào cũng phải đối mặt là: "Ta nên điều bao nhiêu quân?"

Để trả lời câu hỏi này, không thể chỉ nhìn vào số lượng quân đồn trú hiện tại. Một hành tinh có thể đang có 100 quân, nhưng nếu trên bản đồ radar có 2 hạm đội địch, mỗi hạm đội 60 quân đang bay tới và sẽ đáp xuống trong 10 lượt nữa, hành tinh đó có thể sẽ thất thủ. Ngược lại, một hành tinh tiền tuyến chỉ còn 5 quân có thể hoàn toàn an toàn nếu không có mối đe dọa nào trong bán kính 50 lượt bay.

1) **Cơ Chế Garrison Flow Lookahead**: Hệ thống giải quyết bài toán này bằng cơ chế **Garrison Flow Lookahead**. Ý tưởng cốt lõi là duy trì một mảng trạng thái giả lập nội bộ chạy trước dòng thời gian thực tại.

Cho một horizon là số turn mô phỏng cho tương lai gọi là H , giả sử $H = 55$ lượt. Hệ thống khởi tạo một ma trận trạng thái $\mathcal{G}$ kích thước $P \times H$, trong đó $\mathcal{G}_{p,t}$ là số lượng quân đồn trú dự kiến của hành tinh p tại thời điểm t trong tương lai (với $t \in [1, H]$). Tương tự, ta có ma trận $\mathcal{O}_{p,t}$ lưu trữ quyền sở hữu hành tinh.

Quá trình giả lập diễn ra tuần tự từng bước t :

- 1) **Cập nhật kinh tế**: Nếu $\mathcal{O}_{p,t-1}$ không phải là Neutral, hành tinh p sẽ tự động sản sinh thêm quân:

$$\mathcal{G}_{p,t} = \mathcal{G}_{p,t-1} + \rho_p$$

- 2) **Cập nhật giao tranh**: Hệ thống duyệt qua tất cả các hạm đội $f \in \mathcal{F}$ đang bay lơ lửng. Tính toán ETA của f tới đích p . Nếu f dự kiến đáp xuống p đúng vào lượt t , quân số của f sẽ tương tác với $\mathcal{G}_{p,t}$:

- Nếu hạm đội f cùng phe với $\mathcal{O}_{p,t-1}$ (Tiếp viện):

$$\mathcal{G}_{p,t} = \mathcal{G}_{p,t} + s_f$$

- Nếu hạm đội f khác phe với $\mathcal{O}_{p,t-1}$ (Tấn công):

$$\mathcal{G}_{p,t} = \mathcal{G}_{p,t} - s_f$$

- 3) **Cập nhật quyền sở hữu**: Nếu $\mathcal{G}_{p,t} < 0$, điều đó có nghĩa là lực lượng phòng thủ đã bị tiêu diệt hoàn toàn. Hành tinh p chính thức đổi chủ:

$$\mathcal{O}_{p,t} = f.owner$$

$$\mathcal{G}_{p,t} = |\mathcal{G}_{p,t}|$$

2) **Khai thác chỉ số keep_needed**: Lợi ích lớn nhất của bảng mô phỏng $\mathcal{G}_{p,t}$ là việc trích xuất ra chỉ số `keep_needed` cho phe ta. Tại mỗi hành tinh phe ta, ta tìm điểm trung nhất của đồ thị quân số trong H lượt tới:

$$Nadir_p = \min_{t \in [1, H]} (\mathcal{G}_{p,t} \cdot \mathbf{1}_{\{\mathcal{O}_{p,t} == Ta\}} - \mathcal{G}_{p,t} \cdot \mathbf{1}_{\{\mathcal{O}_{p,t} \neq Ta\}})$$

Nếu $Nadir_p \geq 0$, hành tinh p hoàn toàn an toàn trong 55 lượt tới. Nếu $Nadir_p < 0$, hành tinh này chắc chắn sẽ thất thủ, và ta cần phải bổ sung một lượng quân đúng bằng $|Nadir_p|$ ngay từ bây giờ để sống sót.

Giá trị này chính là `keep_needed`. Nhờ có nó, hệ thống có thể tự tin rút 100% quân của một hành tinh hậu phương an toàn (khi `keep_needed = 0`) để ném vào các chiến dịch tấn công quy mô lớn mà không sợ bị đánh lén. Đây là một lợi thế rất lớn so với các bot Heuristic mà em tham khảo, vừa tận dụng quân số để tăng tốc độ, vừa đảm bảo chính xác số quân, đồng thời nhử địch lao tới để lãng phí quân vô ích.

C. KIẾN TRÚC NHIỆM VỤ VÀ TỐI ƯU HÓA KNAPSACK (v1 - v23)

Với nền tảng động lực học và mô phỏng tương lai từ lớp `WorldModel`, bước tiếp theo là xây dựng lõi lập kế hoạch. Phương pháp luận của em coi mỗi kế hoạch là các nhiệm vụ, làm sao để thực thi các nhiệm vụ mà có kết quả tối ưu nhất, cho cả hiện tại và tương lai. Như cái tên của phần này, phần cốt lõi ban đầu được em xây dựng dựa trên DP cho bài toán cái túi, nhưng sau đó nhận ra sự không thích hợp nên đã có những cải tiến nhất định.

1) **Hệ thống 6 lớp nhiệm vụ chiến thuật**: Qua quá trình nâng cấp từ phiên bản v1 đến v23, hệ thống đã hình thành một danh mục gồm 6 loại nhiệm vụ phục vụ các ý đồ chiến thuật chuyên biệt:

- 1) **CAPTURE (Chiếm đóng trung lập)**: Đây là ưu tiên tuyệt đối trong giai đoạn mở màn. Agent xác định các hành tinh chưa có chủ, tính toán lượng tàu cần thiết bằng $Garrison_{neutral} + Margin_{neutral}$, và phóng quân. Lợi nhuận của nhiệm vụ này là tối đa hóa mạng lưới sản xuất tài nguyên.
- 2) **ATTACK (Tấn công địch)**: Đòn đánh trực diện vào hệ thống của đối phương. Điểm đặc biệt của ATTACK là "Lợi ích kép": chiếm được một hành tinh sản lượng ρ của địch đồng nghĩa với việc ta tăng ρ và địch mất ρ , tạo ra sự chênh lệch là 2ρ . Đây là điều em đặc biệt ưu tiên trong đấu 1vs1, còn chế độ 1vs3 thì em sẽ hạn chế lối đánh này để ưu tiên phòng thủ hơn $Margin_{enemy}$.

- 3) **PUSH (Điều quân ra tiền tuyến):** Hành tinh hậu phương an toàn thường tích tụ hàng trăm quân nhưng lại ở quá xa căn cứ địch. Nhiệm vụ PUSH đóng vai trò như một chuỗi cung ứng tự động, chuyển quân tinh từ hậu phương lên các hành tinh tiền tuyến, gọi là frontline, là các hành tinh có nguy cơ giao tranh cao.
- 4) **RELAY (Tiếp sức qua điểm mù - v2):** Để khắc phục điểm mù khổng lồ do mặt trời tạo ra ở trung tâm bản đồ (50, 50), RELAY cho phép một hạm đội bay từ $A \rightarrow M$ (trạm trung gian) rồi từ $M \rightarrow B$. Việc bay theo đường gấp khúc này không những an toàn hơn mà còn giúp hạm đội tận dụng địa sản xuất tại trạm dừng chân M .
- 5) **STEAL (Chiến thuật cướp):** Trong quá trình quan sát, em nhận ra đối với những hành tinh trung lập, ta có thể cạnh tranh vừa chiếm xong, đang yếu nhất thì ta phóng quân qua cướp. Chiến thuật này càng đặc biệt phát huy trong chế độ 1vs3, khi nhiều kẻ địch cùng đánh 1 hành tinh, dựa trên mô phỏng tương lai, quân em sẽ tích trữ quân ở gần đó và phóng cuối cùng để ăn hời. Để thực hiện nhiệm vụ STEAL một cách chính xác, hệ thống sử dụng thuật toán dò tìm thời điểm đánh cắp (Steal Timing Detection) sau đây:

Algorithm 2 Dò tìm Thời điểm Cướp cờ (STEAL Timing Detection)

- 1: **Input:** Hành tinh nguồn của ta P_{src} , Hạm đội đích F_{enemy} đang bay tới hành tinh P_{tgt} .
- 2: $t_{impact} \leftarrow F_{enemy}.ETA$ $\triangleright$ Thời điểm địch đổ bộ
- 3: $G_{tgt_future} \leftarrow \text{SimulateGarrison}(P_{tgt}, t_{impact})$
- 4: **if** $G_{tgt_future} \leq 0$ **then** $\triangleright$ Địch sẽ chiếm được hành tinh này
- 5: $v_{max_speed} \leftarrow \text{fleet_speed}(ships = 1)$ $\triangleright$ Tốc độ tối đa
- 6: $d \leftarrow \text{EuclideanDistance}(P_{src}, P_{tgt})$
- 7: $t_{req_fastest} \leftarrow \lceil d/v_{max_speed} \rceil$
- 8: **if** $t_{req_fastest} \leq t_{impact} + 2$ **then** $\triangleright$ Ta có thể đến ngay sau địch
- 9: **Return** Tạo nhiệm vụ STEAL với ETA mục tiêu $= t_{impact} + 1$
- 10: **end if**
- 11: **end if**
- 12: **Return** None

- 6) **STAGED_RELAY (Tích tụ lực lượng - v14):** Chiến thuật này là do em tham khảo replays của top1 leaderboard. Quân đồng minh dồn quân vào 1 hành tinh để phóng đi tấn công với tốc độ cao và mạnh. STAGED_RELAY hoặc động tương tự, nó đóng vai trò chỉ huy tổng lực. Nó ra lệnh PUSH quân từ nhiều hành tinh lân cận đổ dồn về một nút tập kết. Khi quân số tại nút tập kết vượt qua ngưỡng phòng thủ của đích, hệ thống tự động bắt đầu một cuộc tổng tấn công với số lượng quân lớn, nhanh.

2) **Cơ chế chấm điểm chuẩn hóa Z-Score (Z-Score Standardization):** Để hệ thống tự động đưa ra quyết định, tất cả

các nhiệm vụ ứng viên phải được xếp hạng trên một thang điểm chung. Tuy nhiên, các tiêu chí đánh giá mang đơn vị hoàn toàn khác biệt:

- Lợi nhuận: P (Tàu/Lượt).
- Lượng quân yêu cầu: C (Tàu).
- Thời gian ETA: T (Lượt).
- Mức độ nguy hiểm: θ (Vô hướng).

Việc sử dụng phép cộng tuyến tính ($Score = w_1P + w_2C - \dots$) sẽ gây ra sự nhạy cảm tốt độ. Khi trận đấu bước vào tàn cuộc (Late game), quy mô hạm đội C có thể lên tới hàng ngàn, trong khi P chỉ ở mức 10-20. Điều này sẽ phá vỡ hoàn toàn cấu trúc trọng số.

Thay vào đó, em áp dụng **Chuẩn hóa Z-Score cục bộ (Local Z-Score Normalization)**. Tại mỗi lượt, hệ thống sinh ra một tập hợp hàng nghìn nhiệm vụ ứng viên M . Với mỗi đặc trưng k , ta tính kỳ vọng μ_k và độ lệch chuẩn σ_k trên tập M :

$$\mu_k = \frac{1}{|M|} \sum_{m \in M} X_{m,k}, \quad \sigma_k = \sqrt{\frac{1}{|M|} \sum_{m \in M} (X_{m,k} - \mu_k)^2} \quad (16)$$

Đặc trưng thô X được biến đổi thành đại lượng vô hướng $\hat{X}$:

$$\hat{X}_{m,k} = \frac{X_{m,k} - \mu_k}{\sigma_k + \epsilon}$$

Hàm điểm số tổng hợp (Final Score) được định nghĩa là tổ hợp tuyến tính của các Z-score:

$$Score_m = \alpha \hat{P}_{profit} + \beta \hat{C}_{remain} - \gamma \hat{T}_{time} + \delta \hat{B}_{bonus} - \epsilon \hat{\theta}_{threat} \quad (17)$$

Ngoài ra, hệ thống áp dụng một hàm phạt thời gian phi tuyến. Thay vì phạt tuyến tính, nếu thời gian bay $t > 5$, hình phạt sẽ bùng nổ theo hàm parabol: $f(t) = 5 + (t - 5)^2$. Bởi bất cứ top1 phóng quân với số lượng lớn, nó sẽ bay rất nhanh, phóng thẳng vào giữa quân địch, bị địch bao vây phòng thủ và không làm được gì. Qua quá trình thực nghiệm, em nhận định sau 5 turn bắt đầu phạt là hợp lý nhất.

3) **Phòng thủ chủ động: Bộ Lọc Anti-Vulture (v16-v17):** Để tránh 1 số trường hợp lãng phí quân vô ích trong chế độ 1vs3, em triển khai chiến thuật này. Logic chiến thuật này giả lập kịch bản tương lai: Giả sử ta phóng quân chiếm thành công hành tinh B tại lượt t_{arr} . Hệ thống lập tức quét radar xem địch có hạm đội nào nhắm vào B và đến nơi trong khoảng $[t_{arr}, t_{arr} + 12]$ hay không. Nếu có, và ngân sách của ta không đủ để giữ vững B trong đợt phản công này, nhiệm vụ tấn công gốc sẽ bị hủy bỏ để tránh mất quân vô ích.

Mã giả thuật toán của bộ lọc Anti-Vulture được mô tả như sau:

Algorithm 3 Bộ Lọc Anti-Vulture (Phòng Thủ Chủ Động)

```

1: Input: Ứng viên Nhiệm vụ tấn công  $m$  (từ Nguồn  $A \rightarrow$ 
   Đích  $B$  với thời gian  $T_{bay}$ ), Trạng thái Bản đồ  $\mathcal{S}$ 
2:  $G_{B,future} \leftarrow \text{SimulateArrival}(m) \triangleright$  Số quân của ta còn
   lại tại B sau khi chiếm được
3: for  $f \in \text{EnemyFleets}(\mathcal{S})$  do
4:   if  $f.tgt == B$  then
5:      $t_{enemy\_arrival} \leftarrow f.ETA$ 
6:     if  $t_{enemy\_arrival} \in [T_{bay}, T_{bay} + 12]$  then  $\triangleright$  Dự
       đến ngay sau khi ta chiếm
7:        $G_{B,future} \leftarrow G_{B,future} + \rho_B \times$ 
         ( $t_{enemy\_arrival} - T_{bay}$ )  $\triangleright$  Quân tự sinh ra
8:        $G_{B,future} \leftarrow G_{B,future} - f.ships$   $\triangleright$  Trận
         chiến xảy ra
9:       if  $G_{B,future} \leq 0$  then
10:        Return Kháng nghị (Hủy bỏ nhiệm vụ  $m$ )
11:         $\triangleright$  Sẽ bị cướp lại
12:       end if
13:     end if
14:   end if
15: end for
16: Return An toàn (Tiếp tục nhiệm vụ  $m$ )

```

4) *Giải bài toán cái túi (Knapsack Optimization):* Tập hợp các nhiệm vụ khả thi phải tuân thủ hai ràng buộc nghiêm ngặt:

- 1) **Ràng buộc ngân sách:** Tổng số tàu phóng đi từ hành tinh p không được vượt quá số tàu có sẵn trừ đi `keep_needed`.
- 2) **Ràng buộc xung đột:** Trong cùng một lượt, một hành tinh không được xuất kích nhiều hơn một nhiệm vụ và không có hai nhiệm vụ nào được đánh cùng một đích.

Mô hình hóa bài toán này dẫn đến **Bài toán Cái túi Đa chiều (Multi-dimensional Knapsack Problem - MKP)**. Với ràng buộc thời gian trên Kaggle, thuật toán Quy hoạch động (Dynamic Programming) là không thể. Thay vào đó, em triển khai thuật toán tham lam có ràng buộc tập hợp (Set-Constrained Greedy Selection):

Algorithm 4 Lựa Chọn Nhiệm Vụ Tham Lam Có Ràng Buộc (Set-Constrained Greedy)

```

1: Input: Tập ứng viên  $\mathcal{M}$ , Ngân sách khả dụng  $B[p]$  tại
   mỗi hành tinh  $p \in \mathcal{P}$ 
2:  $\mathcal{M}_{sorted} \leftarrow \text{Sort}(\mathcal{M}, \text{key} = \text{Score}_m, \text{descending} = \text{True})$ 
3:  $U_{src} \leftarrow \emptyset, U_{tgt} \leftarrow \emptyset \triangleright$  Tập hợp Nguồn/Đích đã sử dụng
4:  $Selected \leftarrow \emptyset$ 
5: for  $m \in \mathcal{M}_{sorted}$  do
6:   if  $m.src \in U_{src}$  or  $m.tgt \in U_{tgt}$  then
7:     Continue  $\triangleright$  Bỏ qua để tránh xung đột lệnh trong
       cùng 1 lượt
8:   end if
9:   if  $m.Cost \leq B[m.src]$  then
10:     $Selected.add(m)$ 
11:     $B[m.src] \leftarrow B[m.src] - m.Cost \triangleright$  Trừ đi ngân
       sách đã tiêu
12:     $U_{src}.add(m.src)$ 
13:     $U_{tgt}.add(m.tgt)$ 
14:   end if
15: end for
16: Return  $Selected \triangleright$  Danh sách nhiệm vụ chính thức sẽ
   được phóng

```

Thuật toán này có độ phức tạp $\mathcal{O}(N \log N)$ (chủ yếu nằm ở khâu sắp xếp), đảm bảo hiệu năng thời gian thực xuất sắc.

5) *Siêu tham số đa giai đoạn (Context-Aware Hyperparameters):* Thực nghiệm cho thấy không có một bộ tham số $(\alpha, \beta, \gamma, \delta, \varepsilon)$ nào là "Chén thánh" cho mọi tình huống.

- Chế độ **1v1**: Ưu tiên tiêu diệt đối thủ bằng mọi giá.
- Giai đoạn **Early Game**: Cần ưu tiên Profit (α cao) để phát triển kinh tế.
- Giai đoạn **Late Game**: Cần đánh nhanh thắng nhanh (γ thấp).

Do đó, hệ thống duy trì 4 bộ tham số độc lập và tự động chuyển đổi dựa trên số lượng người chơi còn sống và Step Ratio. Cách tiếp cận thủ công tinh xảo này đã giúp hệ thống ban đầu đạt mốc hơn **1000** điểm, nhưng đồng thời cũng đánh dấu giới hạn vật lý của cấu trúc lập trình hướng đối tượng.

D. BƯỚC NGOẶT: LẬP KẾ HOẠCH TENSOR HÓA (v24)

Khi tiến tới các mức rank cao trên bảng xếp hạng Kaggle, sự yếu kém về mặt tối ưu phần mềm của ngôn ngữ Python bắt đầu bộc lộ rõ rệt. Với cấu trúc kập trình hướng đối tượng (OOP), việc khởi tạo hàng ngàn object `Mission`, và thực thi hai vòng lặp lồng nhau $\mathcal{O}(P^2 \times H)$ để tính toán dự báo Garrison Flow cho từng ứng viên đã khiến agent v23 thỉnh thoảng bị hệ thống cắt bớt mission do Time Limit Exceeded.

Để phá vỡ giới hạn này, ở phiên bản v24, em đã thực hiện một cuộc thay đổi lớn: Đập bỏ toàn bộ mã nguồn OOP và tích hợp, tinh chỉnh một nền tảng mang tên `textttorbit_lite` (được public dưới dạng dataset mã nguồn mở trên Kaggle), dựa hoàn toàn trên **Đại số tuyến tính và Tensor PyTorch**.

1) *Chuyển đổi không gian dữ liệu:* Trong `orbit_lite`, không còn tồn tại các đối tượng đơn lẻ như `planet.x` hay `fleet.ships`. Toàn bộ hệ mặt trời được vector hóa:

- Tọa độ hiện tại được lưu trong ma trận $X, Y \in \mathbb{R}^P$.
- Lượng đòn trú là vector $G \in \mathbb{N}^P$.
- Tốc độ sản xuất là vector $\rho \in \mathbb{R}^P$.

Việc tính toán khoảng cách Euclidean (Distance Matrix) giữa mọi cặp hành tinh $[P \times P]$ giờ đây chỉ cần duy nhất một phép toán vector hóa, tận dụng sức mạnh xử lý SIMD của CPU/GPU, cho tốc độ thực thi chưa tới 1 mili-giây.

2) **Phép chiếu flow-diff quân số 3D (Garrison Flow-diff Projection)**: Thành tựu kỹ thuật lớn nhất của v24 là việc ma trận hóa thuật toán *Garrison Flow Lookahead*. Hệ thống định nghĩa một Tensor động lực học Không-Thời gian $\mathcal{T}$ có kích thước: $\mathcal{T} \in \mathbb{Z}^{P \times T_{horizon} \times K}$. Trong đó P là số lượng hành tinh, $T_{horizon}$ là tầm nhìn dự báo (đang đặt là 55 lượt), và K là tổng số lượng người chơi. Giá trị tại $\mathcal{T}[p, t, k]$ đại diện cho số lượng tàu mà người chơi k nắm giữ tại hành tinh p ở lượt t .

Toán học đằng sau quá trình cập nhật Tensor này chỉ sử dụng hai phép toán nguyên thủy của PyTorch:

- 1) **cumsum (Cộng dồn mảng)**: Dọc theo trục thời gian T , hàm này mô phỏng quá trình sinh quân tự động của hành tinh. Nếu hành tinh thuộc phe k và có sản lượng ρ , giá trị sẽ tăng đều ρ sau mỗi bước t .
- 2) **scatter_add (Rải mảng)**: Để mô phỏng các hạm đội đang bay, hệ thống lấy ma trận ETA của tất cả các chuyển bay, sau đó scatter số lượng tàu s vào chính xác ô không-thời gian (p, t, k) tương ứng.

Chỉ bằng 2 dòng code ma trận, ta đã có một bức tranh toàn cảnh về tương lai của toàn bản đồ mà không cần một vòng lặp for hay while nào. Mã giả cho quá trình chiếu (Projection)⁶ này được thể hiện trong Algorithm 5.

Algorithm 5 Phép Chiếu Dòng Chảy Bằng Tensor (Tensorized Flow Projection)

- 1: **Input**: Tensor Tốc độ sản xuất $\rho \in \mathbb{R}^P$, Tensor Hạm đội đang bay $\mathcal{F}$, Chân trời H .
- 2: Khởi tạo Tensor dự báo $\mathcal{T} \in \mathbf{0}^{P \times H \times K}$
- 3: Khởi tạo Tensor sở hữu hiện tại $\mathcal{O} \in \mathbb{Z}^P$ ▷ Lưu ID người chơi
- 4:
- 5: **Bước 1: Tính lũy kế sản lượng (Vectorized)**
- 6: **for** $t \in [1, H]$ **do**
- 7: $\mathcal{T}[:, t, \mathcal{O}] \leftarrow \rho$ Gán sản lượng cho chủ sở hữu
- 8: **end for**
- 9: $\mathcal{T} \leftarrow \text{cumsum}(\mathcal{T}, \text{dim} = 1)$ Cộng dồn theo trục thời gian
- 10:
- 11: **Bước 2: Rải hạm đội đang bay (Scatter fleets)**
- 12: $t_{ETA} \leftarrow \text{CalculateETA}(\mathcal{F})$ Tensor chứa ETA của mọi hạm đội
- 13: $P_{dest} \leftarrow \mathcal{F}.targets$
- 14: $K_{owner} \leftarrow \mathcal{F}.owners$
- 15: $Ships \leftarrow \mathcal{F}.ships$
- 16: $\mathcal{T}[P_{dest}, t_{ETA}, K_{owner}] \leftarrow \mathcal{T}[P_{dest}, t_{ETA}, K_{owner}] + Ships$
- 17:
- 18: **Return** $\mathcal{T}$ Trả về bức tranh tương lai

3) **Đánh giá đối kháng vòng (Competitive Net-Ship-Delta Scoring)**: Sự ưu việt của Tensor không chỉ dừng ở tốc độ. Khi đã có ma trận $\mathcal{T}$, em nhận ra hàm Z-Score ở v23 đã trở nên lỗi thời. Việc cố gắng cân bằng 5 tham số $(\alpha, \beta, \dots)$ là một sự gượng ép, không mang lại lợi ích.

Trong v24, dựa trên nền tảng lý thuyết trò chơi có tổng bằng không (Zero-Sum Game Theory), em đề xuất hàm **Đánh giá Đối kháng vòng (Competitive Scoring)**. Khi hệ thống xét một hành động tấn công a , nó sẽ sao chép ma trận $\mathcal{T}$ và áp dụng a vào đó, tạo ra một ma trận chênh lệch trạng thái (Flow-diff Tensor).

Từ ma trận này, ta trích xuất được đại lượng $\Delta Net_k(a)$ — sự thay đổi vòng về tổng quân số của người chơi k sau khi hành động a kết thúc (đã trừ hao tổn thất và cộng phần sản xuất giành được). Hàm mục tiêu tối thượng được định nghĩa:

$$Score(a) = \Delta Net_{me}(a) - \sum_{opp \in Opponents} \Delta Net_{opp}(a) \quad (18)$$

Hàm này mang ý nghĩa: "Hành động này mang lại cho phe ta bao nhiêu lợi ích, trừ đi tổng lợi ích mà tất cả kẻ thù khác nhận được". Đoạn code sau mô tả cụ thể nó:

Listing 1: Hàm Chấm điểm Đối kháng vòng trên Tensor

```
def competitive_score(diff: GarrisonFlowDiff, *, player_id: int) -> Tensor:
    """Competitive score: delta_net_me - sum_opp_delta_net_opp."""
    net = diff.net_ship_delta # [*prefix, A]
    me = net[..., int(player_id)]
    opp = net.sum(dim=-1) - me
    return me - opp
```

Sức mạnh của hàm đối kháng vòng nằm ở sự **Thích nghi Không cần tham số (Parameter-free Adaptation)**:

- Trong chế độ FFA, nếu ta mang quân đi đánh đối thủ số 1, cả ta và đối thủ 1 đều hao mòn sinh lực, trong khi đối thủ 2 và 3 ngồi chơi xơi nước và giữ nguyên lực lượng. Điểm số của hành động này sẽ ngay lập tức bị đẩy xuống âm cực sâu, ngăn chặn triệt để lối chơi đổ máu vô ích làm môi cho kẻ thứ ba ngư ông đắc lợi.
- Ở đầu game, hàm sẽ tự động ưu tiên đánh các hành tinh trung lập (vì trung lập không phải là người chơi, nên $\Delta Net_{opp} = 0$, giúp tối đa hóa điểm số).

4) **Thuật toán tham lam**: Với ma trận điểm kích thước $[P \times P]$, việc chạy lại thuật toán Knapsack trên CPU sẽ phá hỏng toàn bộ nỗ lực tăng tốc. Do đó, em triển khai thuật toán chọn lọc trực tiếp trên Tensor:

- 1) Dùng hàm `torch.topk` để lấy ra top các ứng viên có điểm > 0 .
- 2) Khởi tạo một mảng Boolean Mask.
- 3) Áp dụng các toán tử logic bitwise AND / OR để gạt bỏ các ứng viên bị trùng lặp nguồn hoặc đích (Collision resolving) mà không cần vòng lặp.

Kết quả của Tensor hóa là một bước đại nhảy vọt. Tốc độ lập kế hoạch tăng lên **10 lần**. Hệ thống tự do đánh giá hàng trăm ngàn kịch bản phức tạp trong nháy mắt. Phiên bản v24 đã phá vỡ mọi kỷ lục nội bộ, vươn lên đạt **1146.6** điểm và chiếm lĩnh **hạng 625** trên leaderboard.

E. THỬ NGHIỆM TÍCH HỢP TRÍ TUỆ NHÂN TẠO (v25)

Mặc dù v24 đạt hiệu suất cực cao và giải quyết xuất sắc bài toán tối ưu hóa, nó vẫn phụ thuộc hoàn toàn vào một hàm mục tiêu toán học tĩnh. Ở phiên bản cuối cùng (v25), với khao khát vượt qua các thuật toán hard-coded heuristic, em đã tiến hành một thử nghiệm dùng Deep Learning để học các chiến thuật ẩn.

1) *Mạng đánh giá ứng viên*: Ý tưởng của em là đào tạo một mạng Nơ-ron có khả năng nhìn vào một cặp Nguồn-Đích và trực tiếp xuất ra xác suất ($0 \rightarrow 1$) đại diện cho chất lượng của quyết định đó.

Em thiết kế lớp CandidateEvaluatorNet, một mạng Multilayer Perceptron (MLP) thuần túy bằng PyTorch với cấu trúc 4 lớp ẩn: $12 \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow 1$.

Listing 2: Kiến trúc Mạng Nơ-ron Đánh giá Ứng viên v25

```
1 class CandidateEvaluatorNet(torch.nn.Module):
2     def __init__(self, input_dim: int = 12):
3         super().__init__()
4         self.net = torch.nn.Sequential(
5             torch.nn.Linear(input_dim, 64), torch.nn.ReLU(),
6             torch.nn.Linear(64, 64), torch.nn.ReLU(),
7             torch.nn.Linear(64, 32), torch.nn.ReLU(),
8             torch.nn.Linear(32, 1), torch.nn.Sigmoid()
9         )
10    def forward(self, x: Tensor) -> Tensor:
11        return self.net(x).squeeze(-1)
```

2) *Kỹ thuật trích xuất đặc trưng (Feature Engineering)*: Thách thức của mô hình MLP là nó không hiểu được không gian tọa độ thô. Em phải xây dựng một Vector Đặc Trưng 12 chiều (12-D Features) cô đọng nhất:

- (1) **Lợi thế thời gian** ETA chuẩn hóa ($ETA/20.0$).
- (2) **Lực lượng**: Tỷ lệ tàu xuất kích (Send Ratio).
- (3, 4, 5) **Tình trạng Đích**: Quyền sở hữu (+1, 0, -1), Log(Quân đồn trú), và tốc độ sản xuất.
- (6, 12) **Ngữ cảnh ván đấu**: Tỷ lệ Step hiện tại và Số lượng người chơi sống sót.
- (7, 8, 9, 10, 11) **Đặc trưng không gian**: Quy mô hạm đội, Bán kính nguồn, Bán kính đích, Khoảng cách tới mặt trời.

3) *Chăm điểm & Kết Quả*: Để tránh việc mạng AI đề xuất các hành động sai như bay xuyên mặt trời hoặc bay quá xa, em áp dụng kỹ thuật lai ghép (Hybrid):

$$HybridScore = AI_Score \times \left(1.0 - \frac{ETA}{Horizon}\right)$$

Mạng cũng bị khóa trong 40 lượt đầu tiên để đảm bảo tác nhân vẫn chiếm các hành tinh trung lập theo thuật toán tối ưu.

Đánh giá thực nghiệm: Phiên bản v25 đạt **1080.8** điểm, đây là một mức điểm rất tốt, nhưng đáng tiếc lại thụt lùi so với điểm của version trước đó. Phân tích lại code và trận đấu: dữ liệu huấn luyện được em lấy thủ công từ hơn 50 trận đấu thắng của top1 trên kaggle. Em thấy rằng vấn đề là ở mạng MLP bị mù về mặt thời gian; nó không thể cộng dồn quân số tương lai hoàn hảo như phép chiếu Tensor. Nó hay đưa ra những lệnh phóng quân ngay khi còn ít quân, khiến mở đầu

không có quân để chiếm xung quân, tốc độ chậm khiến khi hạm độ đang bay, địch tới chiếm mà bất lực không làm gì được.

VI. PHÂN TÍCH VÀ BÀI HỌC

A. Lỗi Kỹ Thuật Quan Trọng (Pipeline Diep, v5 $\rightarrow$ v6)

Lỗi float($W3 @ x$) khi $W3$ có hình dạng (1, 16) thay vì (16,) khiến toàn bộ mạng không hoạt động và agent chạy ở chế độ heuristic thuần. Sửa bằng `.flat[0]` giải quyết hoàn toàn vấn đề.

B. Phân Bố Công Nhận và Giới Hạn Của Offline RL

Bắt chước Hành vi chuẩn (v5) dùng trọng số thắng/thua cho cả episode, không phân biệt được quyết định tốt/xấu trong từng lượt. IQL+CQL (v8) về lý thuyết giải quyết vấn đề này qua học TD – mỗi quyết định được đánh giá dựa trên $V(s')$ của trạng thái tiếp theo.

Tuy nhiên, kết quả thực tế cho thấy v8 IQL+CQL (543 điểm) lại kém hơn v7 heuristic thuần (589 điểm). Nguyên nhân chính là **distribution shift**: dữ liệu huấn luyện chỉ gồm 400 episode tự chơi với agent v7, khiến mạng Q chưa thấy đủ các trạng thái đa dạng khi đối mặt với agent khác trên bảng xếp hạng. Ngoài ra, không gian hành động biến đổi theo lượt khiến CQL khó ước lượng chính xác phân phối hành động ngoài tập dữ liệu ($\log \sum_a \exp Q(s, a)$ không ổn định khi số ứng viên thay đổi). Kết quả này cho thấy với môi trường có cơ học xác định và không gian hành động không đồng nhất, heuristic có nền tảng lý thuyết vững vẫn cạnh tranh hơn offline RL khi dữ liệu huấn luyện còn hạn chế.

C. Tầm Quan Trọng Của Heuristic Mạnh

Heuristic v7 (ROI + chế độ tiêu diệt) đóng vai trò kép: (1) tạo dữ liệu tự chơi chất lượng cao hơn cho v8; (2) làm bộ điều chuẩn trong kết hợp 50/50, đảm bảo hiệu quả tối thiểu ngay cả khi mạng chưa hội tụ tốt. Việc v7 vẫn vượt v8 trên bảng xếp hạng công khai khẳng định vai trò không thể thay thế của heuristic chất lượng cao trong giai đoạn bootstrapping offline RL.

D. Cải Tiến Liên Tục Là Chìa Khóa (Pipeline An)

Pipeline của An cho thấy mô hình phát triển hiệu quả: xây dựng khung kiến trúc vững (v1), sau đó thêm từng tính năng theo thứ tự ưu tiên – an toàn (v2), bảo toàn tài nguyên (v5), thích nghi theo game mode (v11), thêm mission types mới (v14), và cuối cùng phòng thủ phản đòn (v16–v17). Mỗi cải tiến được kiểm chứng qua submit thực tế trên Kaggle trước khi thêm tính năng tiếp theo.

E. Anti-Vulture – Phòng Thủ Chủ Động

Bộ lọc anti-vulture (v16–v17) minh họa tư duy đối kháng trong RL: không chỉ tối ưu hành động của mình mà còn dự đoán phản ứng của đối thủ. Một nhiệm vụ tấn công có điểm cao vẫn có thể bị loại nếu địch có thể “kèn kèn” (vulture) lại ngay sau khi ta vừa tiêu hao quân.

F. Tối Ưu Tham Số Đa Giai Đoạn (v18–v23)

Việc duy trì 4 bộ tham số riêng (1v1 sớm/muộn, FFA sớm/muộn) thay vì một bộ cố định thể hiện nhận thức rằng chiến thuật tối ưu phụ thuộc cả vào số lượng đối thủ và giai đoạn trò chơi – tương tự curriculum learning trong RL.

VII. KẾT LUẬN

Nhóm 11 tiếp cận bài toán Orbit Wars theo hai hướng độc lập. Pipeline học tăng cường ngoại tuyến (Hoàng Ngọc Điệp, v5–v8) cung cấp khung lý thuyết vững chắc. Pipeline lập kế hoạch của Nguyễn Trường An (v1–v25) đạt kết quả thực tế cao nhất nhóm (**1146.6**, hạng **625**) nhờ sự tiến hóa từ hệ thống Heuristic+WorldModel ban đầu sang kiến trúc Tensor hóa với chấm điểm đối kháng Competitive Scoring và thử nghiệm Mạng nơ-ron.

Bài học chính: (1) kiến trúc tensor hóa cho phép duyệt không gian hành động khổng lồ hiệu quả; (2) Competitive Scoring phản ánh bản chất đối kháng tốt hơn ROI đơn thuần; (3) sự kết hợp giữa WorldModel chính xác và lập kế hoạch quy mô lớn là chìa khóa để đạt thứ hạng cao trong các trò chơi RTS phức tạp.

Hướng phát triển: tích hợp học tăng cường ngoại tuyến để học chính sách từ replay của các version tốt nhất, áp dụng Tìm kiếm Cây Monte Carlo trên WorldModel để nhìn trước sâu hơn, và học tự động tham số đa giai đoạn thay vì tuning thủ công.

REFERENCES

- [1] I. Kostrikov, A. Nair, and S. Levine, “Offline Reinforcement Learning with Implicit Q-Learning,” *International Conference on Learning Representations (ICLR)*, 2022.
- [2] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative Q-Learning for Offline Reinforcement Learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [3] S. An, S. Moon, H. Kim, and H. O. Song, “Uncertainty-Based Offline Reinforcement Learning with Diversified Q-Ensemble,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [4] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, “Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning,” *arXiv preprint arXiv:1910.00177*, 2019.